

Design and Analysis of Algorithms

0/1 Knapsack Optimization using Dynamic Programming

NAME: ADITHYA R

Reg No: 192311369

Assigned Question: Q1. 0/1 Knapsack Optimization

Technique Used: Dynamic Programming (Bottom-Up Tabulation)

1. Problem Statement

A warehouse manager must load items into a truck that has a fixed carrying capacity. Each item has a specific weight and an associated profit, and items cannot be split or reused (0/1 constraint). The manager needs to decide which combination of items to select so that the total profit is maximized without exceeding the weight limit. The problem is solved using Dynamic Programming to determine the optimal profit.

Input Format

- n = number of items
- $\text{Values}[]$ = profit of each item
- $\text{Weights}[]$ = weight of each item
- W = capacity of the knapsack

Sample Input

```
n = 3
Values = 60 100 120
Weights = 10 20 30
W = 50
```

Expected Output

```
Maximum Profit = 220
```

2. Approach / Algorithm

The 0/1 Knapsack problem exhibits optimal substructure and overlapping subproblems, making it a classic candidate for Dynamic Programming. A 2-D table $dp[i][w]$ is built, where $dp[i][w]$ represents the maximum profit obtainable using the first i items with a knapsack of capacity w .

- Base Case: $dp[0][w] = 0$ for all w , and $dp[i][0] = 0$ for all i (no items or no capacity means zero profit).
- Transition: For each item i and capacity w :
 - If $\text{weights}[i-1] > w$, the item cannot be included: $dp[i][w] = dp[i-1][w]$
 - Otherwise, choose the better of excluding or including the item: $dp[i][w] = \max(dp[i-1][w], \text{values}[i-1] + dp[i-1][w - \text{weights}[i-1]])$
- Answer: $dp[n][W]$ holds the maximum achievable profit.

Pseudocode

```
function knapsack(W, weights[], values[], n):
    for i = 0 to n:
        for w = 0 to W:
            if i == 0 or w == 0:
                dp[i][w] = 0
            else if weights[i-1] <= w:
                dp[i][w] = max(values[i-1] + dp[i-1][w-weights[i-1]], dp[i-1][w])
            else:
                dp[i][w] = dp[i-1][w]
    return dp[n][W]
```

3. Program (C)

```
#include <stdio.h>

int max(int a, int b) {
    return (a > b) ? a : b;
}

int knapsack(int W, int weights[], int values[], int n) {
    int dp[n + 1][W + 1];
    int i, w;

    for (i = 0; i <= n; i++) {
        for (w = 0; w <= W; w++) {
            if (i == 0 || w == 0)
                dp[i][w] = 0;
            else if (weights[i - 1] <= w)
                dp[i][w] = max(values[i - 1] + dp[i - 1][w - weights[i - 1]], dp[i - 1][w]);
            else
                dp[i][w] = dp[i - 1][w];
        }
    }

    printf("\nDP Table (rows = items considered, cols = capacity 0..W):\n\n");
    printf("    ");
    for (w = 0; w <= W; w += 5) printf("%4d", w);
    printf("\n");
    for (i = 0; i <= n; i++) {
        printf("i=%d: ", i);
        for (w = 0; w <= W; w += 5) printf("%4d", dp[i][w]);
        printf("\n");
    }

    return dp[n][W];
}

int main() {
    int n, W;
    int values[100], weights[100];

    printf("Enter number of items: ");
    scanf("%d", &n);

    printf("Enter profit (value) of each item:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &values[i]);

    printf("Enter weight of each item:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &weights[i]);

    printf("Enter capacity of knapsack (W): ");
    scanf("%d", &W);

    int maxProfit = knapsack(W, weights, values, n);
```

```

printf("\nMaximum Profit = %d\n", maxProfit);

return 0;
}

```

4. Sample Run and Output

The program was compiled with gcc and executed with the sample input ($n = 3$, Values = 60 100 120, Weights = 10 20 30, $W = 50$). The terminal output below shows the DP table being filled and the final maximum profit.

```

$ gcc knapsack.c -o knapsack
$ ./knapsack
Enter number of items: 3
Enter profit (value) of each item:
60 100 120
Enter weight of each item:
10 20 30
Enter capacity of knapsack (W): 50

DP Table (rows = items considered, cols = capacity 0..W):

      0   5  10  15  20  25  30  35  40  45  50
i=0:   0   0   0   0   0   0   0   0   0   0   0
i=1:   0   0  60  60  60  60  60  60  60  60  60
i=2:   0   0  60  60 100 100 160 160 160 160 160
i=3:   0   0  60  60 100 100 160 160 180 180 220

Maximum Profit = 220

```

Output: Maximum Profit = 220 — this matches the expected output.

5. Complexity Analysis

Metric	Value
Time Complexity	$O(n \times W)$
Space Complexity	$O(n \times W)$ (can be optimized to $O(W)$ with a 1-D array)

There are $n \times W$ subproblems (states in the dp table), and each state takes $O(1)$ time to compute using the recurrence relation, giving an overall time complexity of $O(n \times W)$. The dp table itself requires $O(n \times W)$ space; this can be reduced to $O(W)$ by using a single 1-D array and iterating the weight dimension from high to low.

6. Result

The 0/1 Knapsack problem was successfully solved using Dynamic Programming (bottom-up tabulation). For the given sample input (3 items with values {60, 100, 120}, weights {10, 20, 30}, and capacity 50), the

maximum achievable profit is 220. This is obtained by selecting items 2 and 3 (weights $20 + 30 = 50$, which exactly fills the capacity; profit $100 + 120 = 220$), which matches the value stored in `dp[3][50]`. The program output matches the expected output exactly.